



Institut Teknologi Bandung

Directorate of Sustainable Professional Education

CHAPTER 5

Alat: membuatnya, memilihnya, membatasinya

Alat adalah satu-satunya jalan agen menyentuh dunia — jadi di situlah seluruh kemampuannya, seluruh biayanya, dan seluruh batas izinnya berada.

Duration 3 jam (2 sesi)

Instructor Hendri Karisma

Source Grootendorst & Alammar, An Illustrated Guide to AI Agents — bab 5

Session Objectives

By the end of this session, participants can:

- 1 **Menulis definisi alat** yang bisa dipilih dengan benar oleh model, dan menyebutkan empat cara definisi yang buruk merusak agen.
- 2 **Menghitung ongkos konteks dari daftar alat**, dan menyebutkan kapan menambah alat justru menurunkan kemampuan.
- 3 **Memisahkan alat baca dari alat tulis**, dan menjelaskan kenapa batas izin berada di ketiadaan alat, bukan di prompt.
- 4 **Menjelaskan apa yang dipecahkan MCP** dan apa yang tidak — serta pertanyaan tata kelola yang muncul bersamanya.
- 5 **Menyebutkan tiga cara model belajar memakai alat**, dan mana yang dipakai hampir semua proyek.
- 6 **Merancang penanganan galat alat** yang membuat agen mencoba jalan lain alih-alih mengulang hal yang sama.

SITE

[Course home](#)

REPO

[ai-agentic-demo — enam alat lewat MCP](#)

BOOK

[Maarten Grootendorst & Jay Alammar, An Illustrated Guide to AI Agents \(O'Reilly Media, Early Release\)](#)

01

Alat adalah batas sistemnya

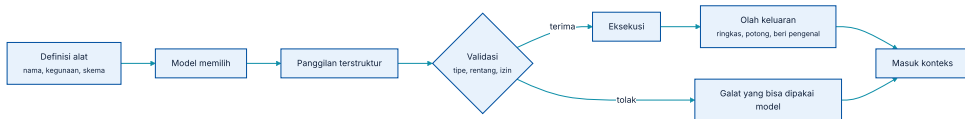
Semua yang bisa dilakukan agen ada di daftar ini. Tidak ada yang lain.

Daftar alat adalah pernyataan kemampuan yang paling jujur

Ini sifat yang jarang dimiliki sistem berbasis model. Untuk hampir semua pertanyaan lain — apakah ia akan berhalusinasi, apakah ia akan patuh — jawabannya statistik. Untuk pertanyaan ini, jawabannya **daftar**.

Dan karena ia daftar, ia bisa **diuji**: `pytest -k test_the_agent_has_no_tool_that_approves_credit` gagal kalau seseorang menambahkan alat tulis kedua. Klaim di slide dan perilaku kode tidak bisa berpisah.

Lima tahap, dan kegagalan bisa terjadi di kelimanya



Dari definisi sampai hasilnya masuk konteks — dengan gerbang di tengah.

Orang cenderung memikirkan \{{\u201cpemanggilan alat\}}\u201d sebagai satu hal. Sebenarnya lima: **mendefinisikan**, **memilih**, **memanggil**, **memvalidasi**, dan **mengolah keluarannya**. Empat dari lima itu kode Anda, dan di situlah sebagian besar perbaikan berada.

02

Mendefinisikan

Deskripsi alat adalah antarmuka, dan pembacanya bukan manusia.

Yang dibaca model hanyalah deskripsinya

Sebut hasilnya, bukan caranya

\{\}u201cmengembalikan rasio kemampuan bayar dari 12 bulan mutasi rekening\{\}u201d — bukan \{\}u201cmenjalankan kueri agregasi\{\}u201d.

Sebut KAPAN memakainya

Satu kalimat tentang kondisi pemakaian menghilangkan sebagian besar panggilan yang salah sasaran.

Tipe ketat, plus satu contoh

Contoh nilai lebih berguna daripada satu paragraf penjelasan tipe.

Sebut apa yang TIDAK dilakukannya

\{\}u201cTidak mengembalikan data pribadi\{\}u201d menutup satu kelas percobaan sebelum terjadi.

Empat cara definisi yang buruk merusak agen

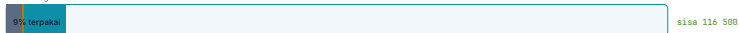
PENYAKIT	GEJALANYA DI JEJAK	OBATNYA
Terlalu umum Dua alat mirip	Alat itu dipanggil untuk hampir semua hal Dipilih bergantian tanpa pola	Persempit namanya dan sebut domainnya Gabungkan, atau bedakan dengan tegas di kalimat pertama
Parameter kabur Tidak menyebut kegagalan	Argumen ngawur, galat di dalam alat Agen mengulang panggilan yang sama	Tipe ketat, enumerasi, contoh nilai Jelaskan bentuk galat yang mungkin kembali

Semua gejala di kolom tengah terlihat seperti **model yang bodoh**, dan hampir semuanya sebenarnya **masalah an-tarmuka yang Anda tulis**

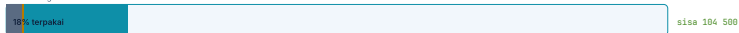
Ongkos yang dibayar tiap giliran, untuk tiap alat

jendela 128k token, dibelanjakan

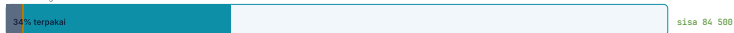
setelah 4 giliran



setelah 10 giliran



setelah 20 giliran



3 500 token sudah terpakai sebelum giliran pertama — dan tiap alat yang ditambahkan memotong lagi.

Jendela 128k dengan enam alat. Langkah 4, 10, dan 20 giliran.

Skema alat adalah bagian **tetap** dari konteks, jadi ia disinggahkan dengan baik — tetapi ia tetap memakan ruang jendela, dan ruang itu tidak bisa dipakai untuk hasil alat maupun riwayat.

Menambah alat bukan penambahan yang netral

Yang bertambah

- ♦ Token skema, tiap giliran
- ♦ Peluang salah pilih di antara alat yang mirip
- ♦ Permukaan yang harus divalidasi dan diamankan
- ♦ Satu hal lagi yang bisa gagal dan harus dipantau

Yang diharapkan

- ♦ Satu kemampuan baru
apakah ini berguna — hampir semuanya berguna — melainkan **kegagalan mana yang sedang saya perbaiki, dan apakah alat yang sudah ada bisa diperluas alih-alih ditambah.**

Karena itu pertanyaan sebelum menambahkan alat bukan

Pola yang berulang: **menggabungkan dua alat kecil jadi satu sering lebih baik daripada mengoptimalkan keduanya**, sebab ia mengurangi giliran sekaligus mengurangi kebingungan memilih.

03

Memilih, memanggil, memvalidasi

Tiga tahap, dan gerbangnya ada di tahap ketiga.

Validasi bukan penanganan galat; ia batas izin

- ① {'h': 'Bentuk dan tipe', 'p': 'Dijamin penyedia kalau memakai skema ketat — tetap periksa, sebab penyedia bisa berubah.'}
- ② {'h': 'Rentang dan keberadaan', 'p': 'Nilai yang masuk akal, pengenalan yang benar-benar ada.'}
- ③ {'h': '**Kepemilikan**', 'p': 'Apakah pengguna INI berhak atas data itu. Ini yang paling sering hilang, dan yang paling mahal hilangnya.'}
- ④ {'h': 'Batas laju dan anggaran', 'p': 'Alat yang sama dipanggil dua puluh kali dalam satu proses adalah gelung, bukan kemajuan.'}

Perhatikan tahap ketiga: izin diperiksa terhadap **pengguna akhir**, bukan terhadap layanan. Agen yang berjalan dengan kredensial layanan bisa membaca segalanya kalau tidak ada yang menyaringnya.

Galat yang berguna membuat agen mencoba jalan lain

Galat yang buruk

- ♦ `Error 500`
- ♦ `null`
- ♦ Melempar pengecualian yang menghentikan proses

Agen tidak tahu apa yang harus berbeda, jadi ia mengulang hal yang sama.

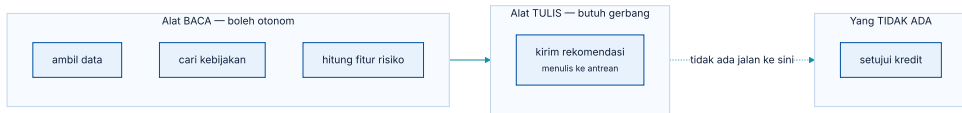
Galat yang berguna

- ♦ `id tidak ditemukan – periksa formatnya: APP-2201`
- ♦ `rentang tanggal maksimum 12 bulan`
- ♦ `butuh hasil analisis dulu; panggil compute_risk_features`

Ketiganya memberi tahu **apa yang harus diubah**.

Ini salah satu perbaikan dengan hasil terbesar per jam kerja di seluruh modul, dan hampir tidak pernah dikerjakan: **tulis pesan galat alat untuk pembaca yang akan mencoba lagi.**

Baca dan tulis bukan risiko yang sama



Lima alat baca, satu alat tulis, dan satu yang sengaja tidak ada.

Alat baca bisa salah dan hasilnya jawaban yang keliru — buruk, tetapi bisa diperbaiki. Alat tulis yang salah meninggalkan **efek**, dan sebagian efek tidak bisa ditarik.

Karena itu pemisahannya bukan gaya penataan kode: alat baca boleh otonom, alat tulis lewat gerbang, dan tindakan yang tidak boleh terjadi **tidak diberi alat sama sekali**. Perubahan sehari, dan keamanan naik jauh.

Mengolah keluaran: pekerjaan yang sering dilewatkan

Kembalikan jawabannya

Rasio yang diminta, bukan 1 843 baris yang menghasilkannya.

Beri pengenalan untuk yang besar

Simpan di luar, kembalikan rujukan-nya, sediakan alat untuk membacanya kalau memang perlu.

Potong dengan batas yang jelas

Dan **katakan** bahwa dipotong — hasil terpotong yang menyamar sebagai hasil lengkap menghasilkan kesimpulan yang salah.

Ketika alatnya terlalu banyak untuk dimuat sekaligus

- ① {'h': 'Kelompokkan menurut tugas, bukan menurut sistem asalnya', 'p': 'Pengguna dan model memikirkan pekerjaan, bukan peta layanan internal Anda.'}
- ② {'h': 'Muat kelompok yang relevan saja', 'p': 'Satu langkah pemilihan kelompok di depan gelung — bisa dengan aturan, bisa dengan pengklasifikasi kecil.'}
- ③ {'h': 'Atau jadikan pencarian alat sebagai alat', 'p': 'Satu alat yang mencari alat lain. Menambah satu giliran, menghemat ribuan token di semua giliran.'}

Perhatikan bahwa ketiganya masalah **pengambilan**, persis seperti Bab 4 — dan itu bukan kebetulan: memilih alat dari lima puluh dan memilih potongan dari lima puluh ribu adalah bentuk masalah yang sama.

Alat yang dibuat agen sendiri

Yang menarik

Tugas yang tadinya butuh lima langkah jadi satu panggilan, dan biaya berikutnya turun jauh.

Yang harus dijawab dulu

Siapa yang meninjau alat itu? Di mana ia berjalan? Apa izin-nya? Alat yang dibuat sendiri oleh sistem adalah **kode baru di produksi tanpa peninjau**.

Bentuk yang bisa dipertanggungjawabkan: agen boleh **mengusulkan** alat, manusia menyetujuinya, dan ia masuk lewat jalur yang sama dengan kode lain. Yang tidak bisa: alat baru langsung bisa dieksekusi.

Alat yang menjalankan kode adalah kelasnya sendiri

Kenapa menggoda

Aritmetika yang pasti, pengolahan data, format yang rapi — semuanya hal yang model buruk mengerjakannya sendiri.

Kenapa berbahaya

Jaringan, berkas, kredensial di lingkungan, dan proses lain di mesin yang sama.

Aturan yang tidak bisa ditawar: **kalau agen menjalankan kode, kode itu berjalan di tempat yang terisolasi** — tanpa jaringan kecuali yang diizinkan, tanpa kredensial, dengan batas waktu dan memori, dan di proses yang boleh dimatikan.

Berlapis, dan jujur soal apa yang tidak ditutupnya

LAPIS	MENAHAN	TIDAK MENAHAN
Daftar impor yang diizinkan	Kesalahan yang tidak disengaja	Kode yang sengaja mencari celah
Batas waktu dan memori	Gelung tak berujung, kehabisan memori	Kebocoran data dalam satu detik
Tanpa jaringan	Pengiriman data keluar	Apa pun yang sudah ada di dalam sandbox
Kontainer / proses terpisah	Sebagian besar hal	Celah pada mesin virtual atau kernel

Kolom ketiga itu yang membuat daftar ini berguna. Sandbox yang dijelaskan sebagai “aman” membuat orang berhenti berpikir; sandbox yang menyebut **apa yang tidak ditutupnya** membuat orang menaruh data sensitif di tempat lain.

Alat tulis harus aman dipanggil dua kali

- ① {'h': 'Beri kunci idempotensi pada tiap panggilan tulis', 'p': 'Panggilan kedua dengan kunci yang sama mengembalikan hasil pertama, bukan membuat entri baru.'}
- ② {'h': 'Kunci keputusan yang sudah diambil', 'p': 'Di demo, keputusan kedua atas antrean yang sama ditolak, dan penolakannya menyebut siapa yang memutuskan pertama.'}
- ③ {'h': 'Jangan mengandalkan agen untuk tidak mengulang', 'p': 'Deteksi pengulangan di anggaran adalah jaring pengaman, bukan jaminan.'}

Alat baca pun punya bentuk kegagalan yang khas

Dipanggil berulang identik

Hampir selalu tanda galatnya tidak informatif, atau hasilnya tidak dipahami. Anggaran pengulangan menangkapnya.

Mengembalikan terlalu banyak

Membanjiri konteks, dan dibayar ulang tiap giliran sesudahnya.

Lambat dan tidak dibatasi

Satu kueri tanpa batas waktu bisa menahan seluruh proses agen. Batas waktu per alat, bukan hanya per proses.

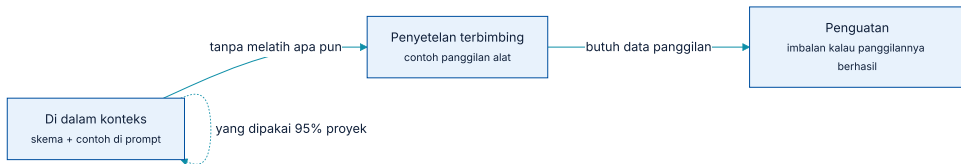
Ketiganya terlihat di jejak dan tidak satu pun menimbulkan galat. Ini pola yang berulang di seluruh modul: **yang merusak sistem agen jarang melempar pengecualian.**

04

Bagaimana model belajar memakai alat

Tiga cara, dan hampir semua proyek memakai yang pertama.

Tiga cara, dengan biaya yang sangat berbeda



Dari yang tidak melatih apa pun sampai yang butuh data panggilan.

Di dalam konteks — skema dikirim bersama permintaan, dan model memakai kemampuan yang sudah dilatihkan padanya. Tanpa pelatihan, tanpa data yang perlu dikumpulkan — dan inilah yang dipakai hampir semua sistem produksi. Dua cara lainnya masuk akal hanya kalau alat Anda sangat khusus **dan** Anda sudah punya ribuan contoh panggilan yang benar.

Apa yang sebenarnya diperbaiki oleh pelatihan

MASALAH	DIPERBAIKI PELATIHAN?	YANG SEBENARNYA BEKERJA
Salah memilih di antara alat mirip	Kadang	Perbaiki deskripsinya dulu — jauh lebih murah
Argumen salah tipe	Ya	Atau: skema ketat, gratis
Lupa memanggil alat sama sekali	Ya	Atau: sebut kondisi pemakaian di deskripsi
Memanggil alat yang tidak ada	Ya	Atau: validasi menolaknya — sudah wajib
Berhenti terlalu awal	Ya	Ini yang paling sulit diperbaiki tanpa pelatihan

Perhatikan kolom ketiga: **empat dari lima punya perbaikan yang tidak butuh melatih apa pun.** Melatih model untuk memakai alat adalah jawaban yang benar jauh lebih jarang daripada yang dikira.

05

Protokol: satu pintu, bukan dua puluh

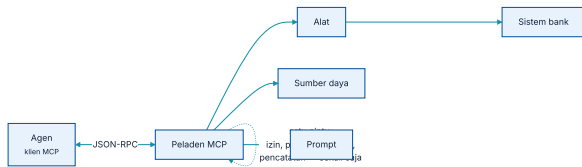
Yang dipecahkan MCP, dan pertanyaan yang dibawahnya.

Masalah yang dipecahkannya adalah masalah perkalian

Protokol mengubah perkalian jadi penjumlahan: tiap sistem menyediakan **satu** peladen, tiap kerangka kerja bicara **satu** bahasa. Ini pola lama — sama seperti driver basis data — dan berhasil karena alasan yang sama.

Perhatikan bahwa ini keuntungan **rekayasa**, bukan keuntungan kecerdasan. MCP tidak membuat agen lebih pintar; ia membuat penyambungannya berhenti jadi pekerjaan yang berlipat.

Satu pintu, dan itu keuntungan keamanan yang sebenarnya



Alat, sumber daya, dan prompt di balik satu peladen.

Kalau semua akses agen ke sistem bank melewati satu peladen, maka **izin, pembatasan laju, dan pencatatan hidup di satu tempat** — ditulis sekali, diaudit sekali, diperbaiki sekali.

Dan pertanyaan tata kelola yang lahir bersamanya, yang lebih baik dijawab sekarang daripada di bawah tekanan: **siapa yang boleh menambahkan alat ke peladen itu?** Itu pertanyaan risiko, bukan pertanyaan teknis.

Tiga hal yang disediakananya, dan bedanya

Alat

Sesuatu yang **dijalankan** dan bisa punya efek. Inilah yang butuh gerbang.

Prompt

Cetakan yang bisa dipakai ulang. Membantu keseragaman; bukan mekanisme keamanan.

Sumber daya

Sesuatu yang **dibaca** — dokumen, berkas, catatan. Tidak punya efek, tetapi tetap bisa membawa perintah yang menumpang.

Pemisahan ini berguna justru saat menilai risiko: **sumber daya tidak perlu gerbang persetujuan, tetapi tetap perlu diperlakukan sebagai masukan yang tidak tepercaya**. Keduanya sering dicampur.

Yang TIDAK dipecahkan protokol

Mutu deskripsi alat

Protokol mengangkut skema; ia tidak membuat skema Anda jelas. Alat yang deskripsinya kabur tetap dipilih dengan salah.

Keputusan izin

Protokol memberi tempat memasang pemeriksaan; ia tidak memutuskan siapa boleh apa.

Perintah yang menumpang di data

Hasil yang dikembalikan peladen tetap masuk ke konteks yang sama dengan perintah.

Ongkos konteks

Skema tetap dikirim tiap giliran. Dua puluh lima alat lewat MCP sama mahalanya dengan dua puluh lima alat tanpa MCP.

Satu detail transport yang menentukan bentuk penyebaran

stdio — klien memiliki prosesnya. Ketika prosesnya mati, alatnya ikut mati. Cocok untuk demo, kelas, dan alat lokal.

HTTP — peladennya berdiri sendiri, bisa diskalakan, dipantau, dan direstart terpisah dari agen yang memakainya. Ini bentuk produksinya.

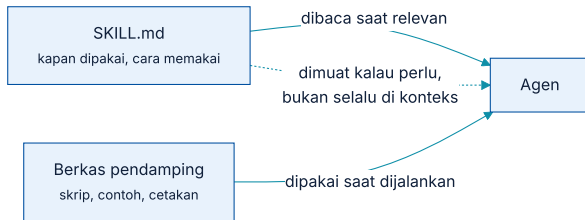
Demo di repo memakai stdio, dan dokumennya menyebut itu dengan jujur beserta akibatnya — sebab **kegagalan yang tidak disebutkan akan ditemukan orang lain pada saat yang paling buruk.**

06

Keterampilan sebagai berkas

Pola yang lebih baru: kemampuan yang dimuat kalau perlu.

Kalau alat itu kata kerja, keterampilan itu prosedur



Satu berkas yang menjelaskan kapan dan bagaimana, plus berkas pendampingnya.

Alat adalah satu tindakan. **Keterampilan** adalah cara mengerjakan sesuatu yang butuh beberapa tindakan, ditulis sebagai berkas — kapan dipakai, bagaimana urutannya, dan berkas pendamping yang dipakai saat menjalankannya.

Kelebihan bentuknya justru pada konteks: **dimuat hanya ketika relevan**. Lima puluh keterampilan tidak memakan ruang jendela, sedangkan lima puluh alat memakan.

Kapan sesuatu pantas jadi keterampilan, bukan alat

KALAU...	JADIKAN	ALASANNYA
Satu tindakan, satu hasil	Alat	Model perlu memanggilnya, bukan membacanya
Prosedur beberapa langkah	Keterampilan	Yang dibutuhkan adalah urutannya, bukan satu panggilan
Jarang dipakai, panjang penjelasannya	Keterampilan	Tidak membebani konteks kalau tidak dipakai
Butuh gerbang persetujuan	Alat	Gerbang dipasang pada eksekusi, dan keterampilan tidak dieksekusi

Baris terakhir yang penting untuk keamanan: keterampilan **menjelaskan** cara melakukan sesuatu, tetapi tetap harus memanggil alat untuk melakukannya — jadi batas izinnnya tidak berpindah.

Bentuknya sederhana, dan itu kekuatannya

Akibat yang baik

- ♦ Ditulis orang yang tahu prosedurnya, bukan hanya programmer
- ♦ Ditinjau seperti dokumen biasa
- ♦ Diversikan di git bersama kode

Akibat yang harus dijaga

- ♦ Teks yang masuk konteks tetap **masuk** tidak **tepercaya**
- ♦ Keterampilan dari sumber luar sama saja dengan menjalankan kode dari sumber luar
- ♦ Tanpa peninjauan, ia jalan pintas ke perilaku baru

Kaidah yang sama seperti alat: **yang menentukan bukan bentuk berkasnya, melainkan siapa yang boleh menambakkannya.**

Alat dirancang untuk agen, bukan disalin dari API yang ada

API YANG ADA	MASALAHNYA BAGI AGEN	ALAT YANG LEBIH BAIK
<code>GET /transactions?page=1</code>	Agen harus mengurus halaman demi halaman	<code>ringkasan_mutasi(id, bulan)</code> — satu panggilan, satu jawaban
Lima titik akhir untuk satu tugas	Lima giliran, lima kesempatan salah	Satu alat yang menyusunnya di sisi peladen
Mengembalikan seluruh objek	Membanjiri konteks	Kembalikan medan yang diminta saja
Galat HTTP mentah	Tidak memberi tahu apa yang harus diubah	Pesan yang menyebut perbaikannya

Aturan praktisnya: **satu alat = satu pertanyaan yang masuk akal ditanyakan manusia**. Kalau agen harus memanggil tiga alat untuk menjawab satu pertanyaan, alatnya yang salah bentuk.

Alat yang mengembalikan bukti, bukan hanya nilai

- 1 { 'h': 'Nilai, sumber, dan waktu', 'p': '"DSCR 0,82, dihitung dari 12 bulan mutasi sampai 2026-07-31" bisa diperiksa. "DSCR 0,82" tidak.' }
- 2 { 'h': 'Versi model atau aturan', 'p': 'Skor yang keluar dari model harus membawa versi modelnya, supaya setahun lagi bisa ditelusuri.' }
- 3 { 'h': 'Pengenal untuk yang besar', 'p': 'Bukan seluruh datanya — rujukan yang bisa dibuka pemeriksa.' }

Inilah yang membuat rekomendasi di demo bisa diperiksa: tiap angka menunjuk panggilan yang menghasilkannya, dan tiap klausul dicetak penuh. Bukan karena promptnya meminta begitu.

Satu alat yang hampir selalu pantas ada

Bertanya kepada manusia

Alat yang tugasnya menghentikan proses dan meminta keputusan. Tanpa ini, satu-satunya cara agen menghadapi ketidakpastian adalah menebak.

Menyerah dengan alasan

Mengembalikan "tidak bisa diselesaikan, karena X" sebagai hasil yang sah. Sistem yang tidak punya cara menyerah akan mengarang.

Keduanya terasa seperti mengakui kekalahan, dan keduanya menaikkan mutu sistem lebih banyak daripada kebanyakan penyeteran prompt. Laju eskalasi yang **nol** hampir selalu tanda ada yang salah, bukan tanda keberhasilan.

Alat bisa diuji tanpa model sama sekali

Uji benar

Masukan sah, keluaran yang diharapkan. Tidak butuh model, tidak butuh jaringan.

Uji tolak

Argumen tidak sah harus ditolak, bukan ditebak maksudnya. Termasuk uji kepemilikan.

Uji struktural

"Tidak ada alat yang bisa menyetujui" dan "alat agen tidak bisa mencapai data pribadi" — diperiksa tiap commit.

Uji ketiga itu yang membedakan kontrol dari klaim. Ia gagal kalau seseorang menambahkan alat yang melanggarnya — termasuk Anda sendiri, enam bulan lagi, sedang terburu-buru.

Yang tidak bisa diuji tanpa model, dan cara menanganinya

- ① {'h': 'Pisahkan dua jenis pengujian sejak awal', 'p': 'Uji mesin (gelung, anggaran, validasi, alat) berjalan luring dan harus selalu hijau. Penilaian model berjalan terpisah dan hasilnya berupa angka, bukan lulus/gagal.'}
- ② {'h': 'Jangan gabungkan keduanya di CI yang sama', 'p': 'Uji yang kadang merah karena modelnya sedang berubah-ubah akan membuat orang berhenti mempercayai seluruh CI.'}
- ③ {'h': 'Pakai penyedia luring untuk uji mesin', 'p': 'Uji yang butuh kunci API adalah uji yang tidak dijalankan orang.'}

07

Praktik

Urutan membangun, dan cara mengetahui alat mana yang bermasalah.

Urutan membangun daftar alat

- ① {'h': 'Mulai dari alat baca saja', 'p': 'Agen yang hanya bisa membaca sudah berguna, dan tidak bisa merusak apa pun sementara Anda belajar bentuk kegagalannya.'}
- ② {'h': 'Tambahkan satu alat tulis, dengan gerbang', 'p': 'Satu. Dan tuliskan alasannya di tempat yang akan dibaca orang lain.'}
- ③ {'h': 'Tuliskan alat yang SENGAJA tidak ada', 'p': 'Beserta ujinya. Ini yang membuat batasnya bertahan sesudah Anda pindah proyek.'}
- ④ {'h': 'Baru pertimbangkan protokol', 'p': 'MCP masuk akal ketika ada lebih dari satu klien atau lebih dari satu sistem. Untuk satu agen dan satu basis data, ia lapisan tambahan tanpa imbalan.'}

Angka per alat yang memberi tahu mana yang bermasalah

ANGKA PER ALAT	KALAU TINGGI BERARTI	TINDAKAN
Laju galat validasi	Skema atau deskripsinya kabur	Perbaiki deskripsi sebelum apa pun
Dipanggil lalu hasilnya tidak dipakai	Salah sasaran	Persempit kondisi pemakaian
Panggilan berulang identik	Galatnya tidak informatif	Tulis pesan galat yang menyebut apa yang harus berubah
Waktu jalan	Alatnya, bukan modelnya	Optimasi di sisi alat; model tidak akan menolong
Ukuran keluaran	Konteks dibanjiri	Ringkas di sisi alat, kembalikan pengenalan

Kelimana per **alat**, bukan per sistem. Rata-rata seluruh sistem hampir selalu menyembunyikan satu alat yang menyebabkan sebagian besar masalah.

Yang dibawa pulang dari bab ini

- ① {'h': 'Daftar alat adalah pernyataan kemampuan', 'p': 'Satu-satunya pertanyaan tentang sistem berbasis model yang jawabannya daftar, bukan statistik.'}
- ② {'h': 'Deskripsi alat adalah antarmuka', 'p': 'Sebagian besar yang terlihat seperti model bodoh adalah antarmuka yang ditulis buruk.'}
- ③ {'h': 'Menambah alat tidak netral', 'p': 'Token tiap giliran, kebingungan memilih, dan permukaan yang harus diamankan — untuk satu kemampuan.'}
- ④ {'h': 'Baca dan tulis dipisah; yang terlarang tidak diberi alat', 'p': 'Batas yang bisa dibujuk bukan batas.'}
- ⑤ {'h': 'Galat alat ditulis untuk pembaca yang akan mencoba lagi', 'p': 'Perbaiki dengan hasil terbesar per jam kerja, dan hampir tidak pernah dikerjakan.'}